Unit-03

2D and 3D Transformation

3.1 2D and 3D Transformations: Translation, Rotation (about origin and arbitrary point), Scaling (about origin and arbitrary point), Reflection and Shear

Definition and Types of Transformations:

Transformations are mathematical operations that modify the position, size, orientation, or shape of an object in a coordinate system. Transformations are categorized as:

1. 2D Transformations: Applied in a 2D plane.
2. 3D Transformations: Applied in a 3D space.

2D Transformations:

1. Translation:

Translation is a transformation that shifts an object from one position to another in the coordinate space without altering its size, shape, or orientation.

Formula:

$$x' = x + T_x,$$

$$y' = y + T_y$$

Where T_x and T_y are translation distances along the x and y axes.

2. Rotation:

Rotation is a transformation that pivots an object around a fixed point (e.g., the origin or an arbitrary point) by a specified angle, without changing its size or shape.

About the Origin:

$$x' = x \cos \theta – y \sin \theta,$$

$$y' = x \sin \theta + y \cos \theta$$

- About an Arbitrary Point (x_r, y_r):

1. Translate (x_r, y_r) to the origin.

2. Apply rotation.
3. Translate back to (x_r, y_r).

3. Scaling:

Scaling is a transformation that changes the size of an object with respect to a reference point (usually the origin) while maintaining its shape and proportions.

Formula:

$$x' = S_x \cdot x,$$

$$y' = S_y \cdot y$$

Where S_x and S_y are scaling factors.

4. Reflection:

Reflection is a transformation that creates a mirror image of an object relative to a reference line in 2D or a reference plane in 3D.

Formulas:

- About x-axis: $x' = x,$ $y' = -y$
- About y-axis: $x' = -x,$ $y' = y$
- About origin: $x' = -x,$ $y' = -y$

5. Shear:

Shearing is a transformation that distorts the shape of an object by shifting its coordinates along one axis, while the other axis remains unchanged. The degree of distortion is determined by a shear factor.

Formulas:

- x-direction shear: $x' = x + Sh_x \cdot y,$ $y' = y$
- y-direction shear: $y' = y + Sh_y \cdot x,$ $x' = x$

3D Transformations:

1. Translation:

$$x' = x + T_x,$$

$$y' = y + T_y,$$

$$z' = z + T_z$$

2. Rotation:

- About x-axis: $y' = y \cos\theta - z \sin\theta$,
  $z' = y \sin\theta + z \cos\theta$
- About y-axis: $x' = x \cos\theta + z \sin\theta$
  $z' = -x \sin\theta + z \cos\theta$
- About z-axis: $x' = x \cos\theta - y \sin\theta$,
  $y' = x \sin\theta + y \cos\theta$

3. Scaling:

$$x' = S_x \cdot x,$$

$$y' = S_y \cdot y,$$

$$z' = S_z \cdot z$$

4. Reflection:

1. Across xy-plane: $z' = -z$
2. Across yz-plane: $x' = -x$
3. Across xz-plane: $y' = -y$

5. Shear:

In 3D, shear transformations distort the object along one axis by a proportion of the other two axes.

1. **Shear in X-direction:**

$$x' = x + sh_{xy} \cdot y + sh_{xz} \cdot z,$$

$$y' = y,$$

$$z' = z$$

Matrix Representation:

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & sh_{xy} & sh_{xz} & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

2. **Shear in Y-direction:**

$$x' = x,$$
$$y' = y + sh_{yx} \cdot x + sh_{yz} \cdot z,$$
$$z' = z$$

Matrix Representation:

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ sh_{yx} & 1 & sh_{yz} & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

3. **Shear in Z-direction:**

$$x' = x,$$
$$y' = y,$$
$$z' = z + sh_{zx} \cdot x + sh_{zy} \cdot y$$

Matrix Representation:

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ sh_{zx} & sh_{zy} & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

4. **Combined Shear in X, Y, and Z directions:**

$$x' = x + sh_{xy} \cdot y + sh_{xz} \cdot z,$$
$$y' = y + sh_{yx} \cdot x + sh_{yz} \cdot z,$$
$$z' = z + sh_{zx} \cdot x + sh_{zy} \cdot y$$

Matrix Representation:

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & sh_{xy} & sh_{xz} & 0 \\ sh_{yx} & 1 & sh_{yz} & 0 \\ sh_{zx} & sh_{zy} & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

3.2 Representation of 2D and 3D Transformations in Homogeneous Coordinates:

Homogeneous coordinates are an extended coordinate system used in computer graphics to simplify the computation of transformations, such as translation, scaling, rotation, and perspective projection. This system introduces an additional dimension to the coordinates, enabling matrix-based operations.

- A 2D point (x, y) is represented in homogeneous coordinates as (x,y,1). This allows translation and other transformations to be performed using a unified matrix representation.

Transformation Matrices in 2D

1. **Translation: Moves the point by (t_x, t_y):**

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

2. **Scaling: Scales the point by factors (s_x, s_y):**

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

3. **Rotation: Rotates the point by an angle θ about the origin.**

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- A 3D point (x, y, z) is represented in homogeneous coordinates as (x, y, z, 1). This allows 3D transformations to be represented with 4x4 matrices.
  Transformation Matrices in 3D

1. **Translation**: Moves the point by (t_x, t_y, t_z).

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

2. **Scaling**: Scales the point by factors (s_x, s_y, s_z).

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

3. **Rotation**: Rotates the point about one of the coordinate axes (x, y, or z).

- About the x-axis:

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

- About the y-axis:

$$\begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- About the z-axis:

$$\begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3.3 Successive and Composite Transformations

Transformations are fundamental operations in computer graphics for manipulating the position, orientation, and size of objects in a scene. Successive transformations and composite transformations provide efficient methods to achieve complex changes to objects.

Successive Transformations

In successive transformations, multiple transformations (e.g., translation, rotation, scaling) are applied to an object **one after another**, in sequence.

Example:

1. **Translation**: Move the object to a new position.
2. **Rotation**: Rotate the object about a specified point.
3. **Scaling**: Adjust the size of the object.

For each transformation, the object's coordinates are updated, and the result of one transformation becomes the input for the next.

Step-by-Step Process:

1. Apply the first transformation (e.g., translation) to the object's coordinates.
2. Use the transformed coordinates as the input for the second transformation (e.g., rotation).
3. Repeat this process for all desired transformations.

Pros:

- Allows detailed control over the sequence of operations.
- Useful when the order of transformations matters, such as rotating an object **after** moving it to a specific position.

Example Calculation:

For an object with an initial coordinate (x, y):

1. Translate it by (T_x, T_y):

$$x_1 = x + T_x, y_1 = y + T_y$$

2. Rotate the result by an angle θ:

$$x_1 \cos\theta - y_1 \sin\theta, y_2 = x_1 \sin\theta + y_1 \cos\theta$$

3. Scale the result by S_x, S_y:

$$x' = x_2 \cdot S_x, y' = y_2 \cdot S_y$$

Composite Transformations

In composite transformations, multiple transformation matrices are combined into a single matrix, which can then be applied to an object in **one operation**. This eliminates the need for step-by-step application of each transformation.

Matrix Representation:

Each transformation (translation, rotation, scaling, etc.) can be represented as a matrix. To create a composite transformation:

1. Multiply the individual transformation matrices in the order of application.
2. Use the resulting composite matrix to transform the object.

General Formula:

If M_1, M_2, M_3…, M_n are the transformation matrices, the composite matrix M_c is given by:

$$M_c = M_n \cdot M_n - 1 \cdot \ldots \cdot M_1$$

Here, the order of matrix multiplication is critical, as matrix multiplication is not commutative.

Benefits of Composite Transformations:

- **Efficiency**: Combining transformations into a single matrix reduces the number of operations required.
- **Simplicity**: Applying one composite matrix is simpler and faster than applying each transformation individually.
- **Consistency**: Ensures transformations are applied in the exact intended sequence.

3.4 Window to Viewport Transformations:

The **Window to Viewport Transformation** maps a portion of the world coordinate system (referred to as the **window**) to a portion of the display area (referred to as the **viewport**). This transformation is essential in computer graphics for visualizing a specific part of the scene on the screen.

Formula:

$$x_v = x_w \cdot \frac{x_{v_{max}} - x_{v_{min}}}{x_{w_{max}} - x_{w_{min}}} + x_{v_{min}}$$

$$y_v = y_w \cdot \frac{y_{v_{max}} - y_{v_{min}}}{y_{w_{max}} - y_{w_{min}}} + y_{v_{min}}$$

3.5 2D and 3D Viewing Pipeline:

The **Viewing Pipeline** describes the sequential steps for converting a 3D object in a virtual scene to a 2D representation on a display screen. Each stage plays a critical role in rendering graphics accurately.

Modelling Transformation: Converts object coordinates to world coordinates.

1. Viewing Transformation:
   Converts world coordinates to viewing coordinates.
2. Projection Transformation:
   Converts 3D viewing coordinates to 2D screen
3. coordinates.
4. Clipping:
   Removes objects outside the visible region.
5. Viewport Transformation:
   Maps the 2D image to device coordinates.